CSE 3105 / CSE 3137

OBJECT-ORIENTED ANALYSIS AND DESIGN

FALL 2025

COURSE PROJECT: *BagItUp!*

Requirements Analysis Document

Group 19

Zahra Ismayilli	*220315094*
Mehmet Mert Yiğit	*220315103*
Begüm Sezer	*220315067*
Zeynep Reyyan Kaleli	*220315071*
Göktuğ Kayacı	*230315009*
Aykhan Bakhshizada	*210315091*

7 December 2025

Table of Contents

1 Introduction

The BagItUp! system is a mobile application designed to combat the global issue of food waste by creating a digital marketplace where consumers can purchase unsold food from local businesses at a heavily discounted price. The system facilitates a win–win relationship between food providers (restaurants, bakeries, cafes, and grocery stores) and consumers, ensuring that surplus food is efficiently redistributed rather than discarded.

The project was developed in response to the increasing need for sustainable food management solutions and eco-conscious consumer practices. Many existing food service providers lack a simple, unified platform that allows them to quickly sell surplus food in real time, while consumers seek affordable options aligned with environmental responsibility. BagItUp! directly addresses this gap by combining geolocation technology, secure digital transactions, and real-time stock management.

The scope of the system includes two primary user groups:

Businesses, who can list surplus “Surprise Bags” containing leftover food items near the end of the business day.

Consumers, who can discover nearby offers, make pre-payments, and verify pickup or delivery using a digital code.

Development of this system was guided by feasibility studies of similar sustainability-based platforms such as Too Good To Go and Olio, identifying opportunities to improve local delivery options, loyalty rewards, and merchant control.

The objectives of BagItUp! are as follows:

To minimize food waste by enabling real-time sale of surplus food through a mobile platform.

To provide businesses with a simple, maintainable dashboard to list, manage, and verify bag collections.

To offer consumers a secure, transparent, and rewarding experience when purchasing surplus food.

To introduce an eco-points system that encourages loyalty and sustainable consumption behavior.

Success criteria for the project include:

Seamless and secure pre-payment functionality.

Average response time under 2 seconds for nearby bag listings.

Positive user feedback on ease of use and environmental contribution.

Demonstrable reduction in food waste among participating businesses.

BagItUp! aims to become a reliable digital bridge between businesses with surplus inventory and consumers seeking value and sustainability, thereby promoting a circular economy model in the food sector.

2 Current System

Today, several digital solutions have emerged to address the issue of food waste. Prominent examples include Too Good To Go, Olio, and Karma, which connect consumers with local businesses offering surplus food at discounted prices. These platforms allow food providers to list unsold items at the end of the day, enabling consumers to purchase them affordably—thereby achieving both economic and environmental benefits.

However, despite their success and growing adoption, the current systems still exhibit several limitations and gaps:

Lack of localization:

Existing platforms are primarily concentrated in large metropolitan areas, leaving small businesses and local communities underserved or excluded from such systems.

Limited business control:

In most current applications, businesses have restricted control over features such as promotions, campaign customization, or delivery options. This limits their ability to build customer loyalty or adapt the system to their unique operational needs.

Insufficient loyalty and incentive mechanisms:

Current systems focus mainly on discounted transactions but lack a structured reward system to encourage sustainable behavior. As a result, users engage for short-term savings rather than long-term environmental impact.

Limited delivery and accessibility options:

Many existing applications only support “in-store pickup,” offering no local delivery features—even within short distances. This reduces accessibility for users with limited time or mobility.

Therefore, while current systems have made significant contributions toward reducing food waste, they lack comprehensive localization, flexible business autonomy, and sustainability-driven engagement.

The proposed BagItUp! system aims to fill these gaps by offering a localized, user- and business-centered platform that supports both pickup and delivery, integrates an eco-point reward system, and empowers local food providers to actively participate in the circular food economy.

3 Proposed System

The proposed BagItUp! system will provide an end-to-end digital solution to the food waste problem by creating a platform where businesses can list their surplus items as “Surprise Bags,” and consumers can discover, pay for, and collect these items seamlessly.

The system will consist of two integrated components:

Mobile Application (Consumer-Facing): Enables users to browse available Surprise Bags based on real-time location data, view details such as price and pickup time, and make secure payments directly within the app. After purchase, the system generates a unique digital verification code (QR code) that serves as proof of payment during pickup or delivery.

Business Dashboard (Provider-Facing): A web-based or in-app interface that allows businesses to quickly list available bags, monitor ongoing sales, manage promotions, and verify collections using the consumer’s QR code.

Additionally, the system will include an eco-points reward mechanism, allowing consumers to accumulate points with each purchase that can later be redeemed for discounts or coupons.

The core goals of the proposed system are:

To automate surplus food redistribution, reducing waste at its source.

To enhance transparency and trust through secure payments and digital verification.

To motivate sustainable behavior through gamified eco-rewards.

To support business efficiency with simple, maintainable, and scalable technology.

In the subsequent sections, detailed Functional Requirements, Nonfunctional Requirements, and System Models will describe the technical and behavioral specifications that ensure the system meets these goals effectively.

3.1 Functional Requirements

Functional requirements describe the interactions between the system and its environment, detailing what the system must do.

FR-01: Surprise Bag Listing: The Business must be able to easily list the daily quantity of surplus bags ("Surprise Bags") available and set a fixed, heavily discounted price for them.

FR-02: Location-Based Discovery: The system must use geolocation to show the Consumer all nearby available Surprise Bags on a map to ensure convenience and facilitate quick matching.

FR-03: Secure Pre-Payment: The Consumer must be able to securely pre-pay for the Surprise Bag via the app.

FR-04: Digital Verification Code: The system must generate a unique digital collection code/QR code for the Consumer after payment for in-person collection verification.

FR-05: Collection Approval: The Business must be able to approve the collection using the Consumer's digital code via a streamlined dashboard.

FR-06: Delivery Feature Management (Optional): The Business must be able to enable an optional local delivery feature for a small additional fee, specifying the delivery radius (e.g., 2–3 km).

FR-07: Eco-Point Accumulation: The system must track purchases and automatically award the Consumer "eco-points" with each purchase.

FR-08: Coupon and Promotion System: The system must allow points to be converted into coupons or discounts, and allow the Business to create promotional campaigns (e.g., "Buy 3 Bags → Get 10% Off Next").

3.2 Nonfunctional Requirements

Non-functional requirements describe user-level requirements that are not directly related to functionality, covering aspects like usability, reliability, and performance.

Usability (NFR-01, NFR-02)

The mobile application's interface must be user-friendly and designed for effortless in-person collection.

The application must clearly display the collection window (e.g., a narrow, designated time) and delivery availability/pricing before checkout.

Performance (NFR-03, NFR-04)

The response time for loading the map with nearby available bags must be less than 2 seconds.

The system must reliably handle concurrent searching and purchasing activities during the peak collection window (e.g., the hour before closing time).

Reliability (NFR-05)

The pre-payment system must ensure that no financial transaction results in data loss or incorrect fund allocation in case of a system failure.

Implementation (NFR-06 - Constraint)

The application must be developed as a cross-platform mobile application supporting both iOS and Android operating systems.

Security (NFR-07)

All pre-payment transactions must comply with industry-standard security protocols.

Supportability (NFR-08)

The Business dashboard must be designed to be easily maintainable and allow the addition of new promotional campaign types without major core system modifications.

3.3 System Models

3.3.1 Scenarios

3.3.1.1 Scenario 1 – Consumer Buys and Collects a Surprise Bag (Critical)

Deniz is a registered and logged-in user. It is 6:30 PM, and a nearby bakery has listed Surprise Bags available for collection between 6:30 PM and 7:00 PM. Deniz opens the app and views nearby businesses on the map (FR-02). She selects the bakery, checks the price, and chooses the “In-store Pickup” option. She completes the secure in-app payment (FR-03). The system generates a unique digital QR code and a receipt (FR-04). At 6:50 PM, Deniz arrives at the bakery and presents the QR code to the cashier. The bakery employee scans the code using the business dashboard and approves the collection (FR-05). Deniz receives the prepared Surprise Bag.The system notifies her that the order is complete and adds eco-points to her account (FR-07). Later, Deniz can view her total eco-points in the profile section and use them to redeem future discounts.The bakery may create special loyalty campaigns that reward returning customers like Deniz with extra eco-points or price reductions for repeat purchases.

3.3.1.2 Scenario 2 – Purchase with Optional Delivery

A Consumer prefers to have their Surprise Bag delivered instead of collecting it personally.
The Consumer finds a suitable business in the app and proceeds to the payment screen.
They select the optional “Local Delivery” option and agree to pay a small extra fee (FR-06).
The Business confirms the order and assigns a Courier or a store employee for delivery.
The Courier checks the assigned order in the app, picks up the bag, and delivers it to the Consumer’s address.
Upon delivery, the Courier scans the digital QR code in the app to confirm the completion.
The system marks the order as “Delivered” and sends a confirmation notification to the Consumer (FR-05).

3.3.1.3 Scenario 3 – Business Lists Bags

A Business wants to list its unsold food items at the end of the day as discounted Surprise Bags.
The Business employee logs into the management dashboard of the app.
They enter “8” in the “Available Bag Quantity” field and click the “Publish” button (FR-01).
Within seconds, the new Surprise Bags become visible on the Consumer’s map (FR-02).
Consumers can now view and purchase the newly listed bags directly from the app.

3.3.2 Use Case Model

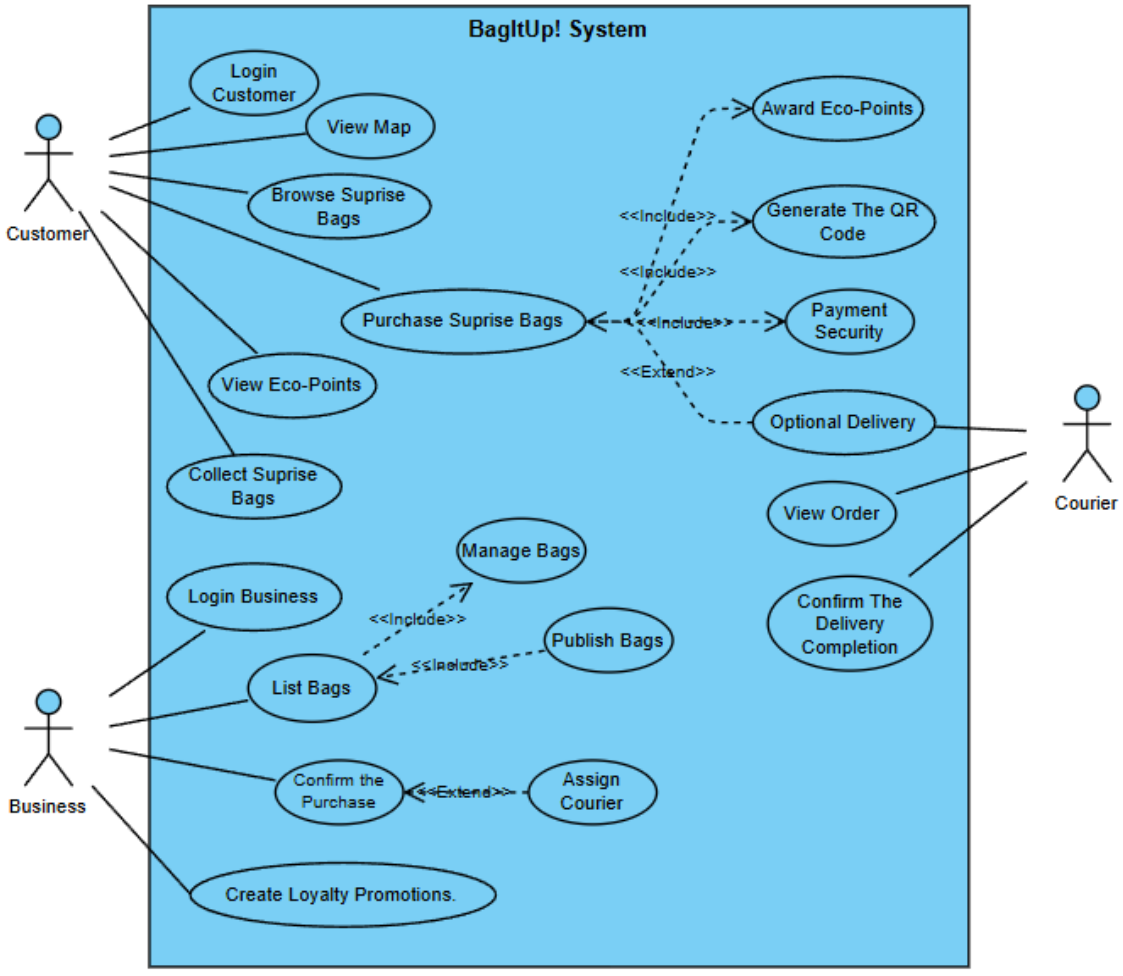

Figure 3.3.2.1 UML Use Case Diagram of BagItUp! System

3.3.2.1 Use Case 1 – Purchase and Collect Surprise Bags

Name: Purchase and Collect Surprise Bags

Participating Actors: Consumer (Deniz), Business (Bakery Employee)

Entry Condition: The Consumer is logged in, and at least one Business has available Surprise Bags within the active collection window.

Flow of Events:

1. The Consumer browses nearby listings via map view.
2. Selects a Business, reviews details, and chooses in-store pickup.
3. Completes secure in-app payment.
4. The system generates a QR pickup code and receipt.
5. The Consumer presents the code at the Business.
6. The Business scans and confirms the collection.
7. The system updates the order as completed and awards eco-points to the Consumer.
8. The Consumer can view and redeem eco-points for future discounts through the app's loyalty feature.
9. The Business can create loyalty campaigns that encourage repeat purchases.

Exit Condition: The Surprise Bag is successfully purchased and collected; eco-points are credited and stored in the Consumer's profile.

Exceptions:

1. Payment gateway error → user retries or cancels.
2. Out-of-stock at payment → automatic refund triggered.
3. Code invalid or expired → manual verification by Business.

Nonfunctional Requirements:

1. NFR-U1: The entire purchase process should take ≤ 3 screens.
2. NFR-R2: Order and loyalty data must be stored reliably to prevent loss during crashes.
3. NFR-S1: All transactions and loyalty updates are logged for audit and reward tracking.

3.3.2.2 Use Case 2 – Purchase with Optional Delivery

Name: Purchase with Optional Delivery

Participating Actors: Consumer, Business, Courier

Entry Condition: The Consumer has placed an order and selected the delivery option during payment.

Flow of Events:

1. The Consumer confirms delivery with an added fee.
2. The Business receives a delivery request and assigns a Courier.
3. The Courier views the order in the app and collects the bag.
4. The Courier delivers it to the Consumer's address.
5. Upon delivery, the Courier scans the QR code to confirm completion.

Exit Condition: The Surprise Bag is delivered successfully; the order is marked as completed.

Exceptions:

1. Courier unavailable → fallback to in-store pickup.
2. Delivery delayed → Consumer notified automatically.

Nonfunctional Requirements:

1. NFR-P1: Delivery status updates must appear in ≤2 seconds.
2. NFR-R1: 99.5% uptime for real-time order tracking.
3. NFR-S1: Delivery logs stored for each completed transaction.

3.3.2.3 Use Case 3 – Business Lists Bags

Name: Business Lists Bags

Participating Actors: Business (Food Provider)

Entry Condition: Business account is authenticated and surplus items are available.

Flow of Events:

1.The Business logs into the management dashboard.

2.Enters available bag quantity, sets price and collection window.

3.Confirms and publishes the listing.

4.Bags instantly appear on the Consumer's map view.

Exit Condition: The Surprise Bags are successfully listed and visible to Consumers.

Exceptions:

1.Missing input fields (price, quantity) → system prompts correction.

2.Network failure → listing saved locally and auto-published later.

Nonfunctional Requirements:

1.NFR-U1: Creating a listing must take less than 2 minutes.

2.NFR-R1: Listings visible to Consumers within 3 seconds after publishing.

3.NFR-S1: All listing data backed up automatically to the server.

3.3.3 Object Model

The object model defines the static structure of the BagItUp! system by identifying the domain classes, their attributes, operations, and associations. These classes were derived from the functional requirements and critical use cases.

Below is the UML Class Diagram and textual explanations of the important objects.

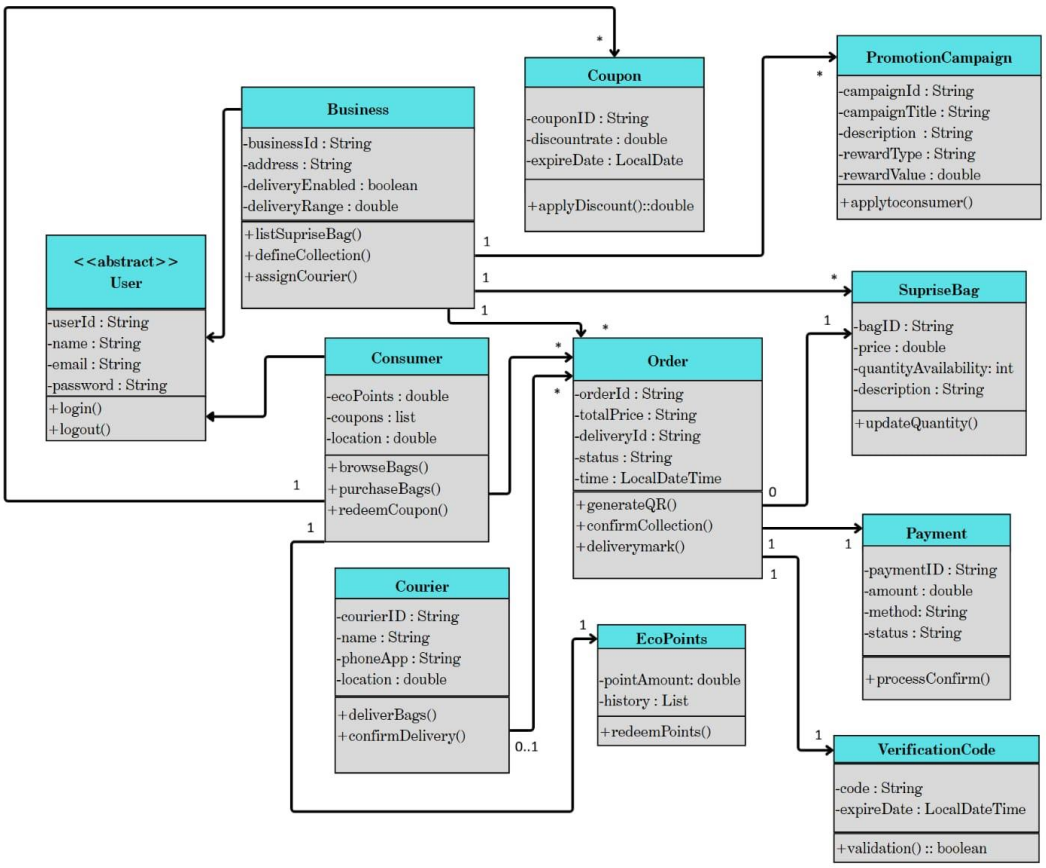

Figure 3.3.3.1 UML Class Diagram of the BagItUp! System

3.3.3.1 Classification of Objects

1. **Entity Objects**

- User (abstract)
- Consumer
- Business
- Courier
- SurpriseBag
- Order

- Payment
- EcoPoints
- Coupon
- PromotionCampaign
- VerificationCode

2. Boundary Objects

- MobileApp (Consumer-facing)
- BusinessDashboard (Business-facing)
- Payment Gateway UI
- Delivery Assignment Interface

3. Control Objects

- OrderController
- PaymentController
- CouponController
- DeliveryController
- AuthenticationController

3.3.3.2 Textual Description of Key Classes

User (Abstract Entity)

Represents any registered account in the system (Consumer, Business employee, Courier)

Attributes: userId, name, email, password

Operations: login(), logout()

Consumer

A user who browses, purchases, and collects Surprise Bags.

Attributes: ecoPoints (double), coupons (list), location

Operations: browseBags(), purchaseBags(), redeemCoupon()

Business

A food provider offering Surprise Bags.

Attributes: businessId, address, deliveryEnabled, deliveryRange

Operations: listSurpriseBag(), defineCollection(), assignCourier()

SurpriseBag

A surplus food package prepared by the Business.

Attributes: bagID, price, quantityAvailability, description

Operations: updateQuantity()

Order

Represents a placed purchase.

Attributes: orderId, totalPrice, deliveryId, status, time

Operations: generateQR(), confirmCollection(), deliveryMark()

Payment

Handles secure in-app financial processing.

Attributes: paymentID, amount, method, status

Operations: processConfirm()

Coupon

Contains discount information awarded through eco-points or campaigns.

Attributes: couponID, discountRate, expireDate

Operations: applyDiscount()

PromotionCampaign

Created by Businesses to encourage loyalty.

Attributes: campaignId, campaignTitle, description, rewardType, rewardValue

Operations: applyToConsumer()

VerificationCode

A time-limited QR code used to validate collection/delivery.

Attributes: code, expireDate

Operations: validation()

3.3.3.3 Relationships Overview

- A Consumer can place *many* Orders (1..*).
- A Business can list *many* SurpriseBags (1..*).
- An Order is linked to exactly one Payment.
- A PromotionCampaign may apply to *multiple* Consumers.
- A Courier may deliver *zero or many* Orders (0..1).
- A Consumer can hold *multiple* Coupons.

This structure ensures correct representation of purchase flow, collection verification, eco-rewards, and business management.

3.3.4 Dynamic Models

The dynamic model describes how objects interact during runtime. BagItUp! involves three critical operational flows:

1. **Purchase & Pickup**
2. **Purchase with Delivery**
3. **Business Listing Flow**

Sequence diagrams and the Order State Machine Diagram illustrate these behaviors.

3.3.4.1 Sequence Diagram 1 : Consumer Purchases and Collects a Bag

This diagram represents the most critical use case.

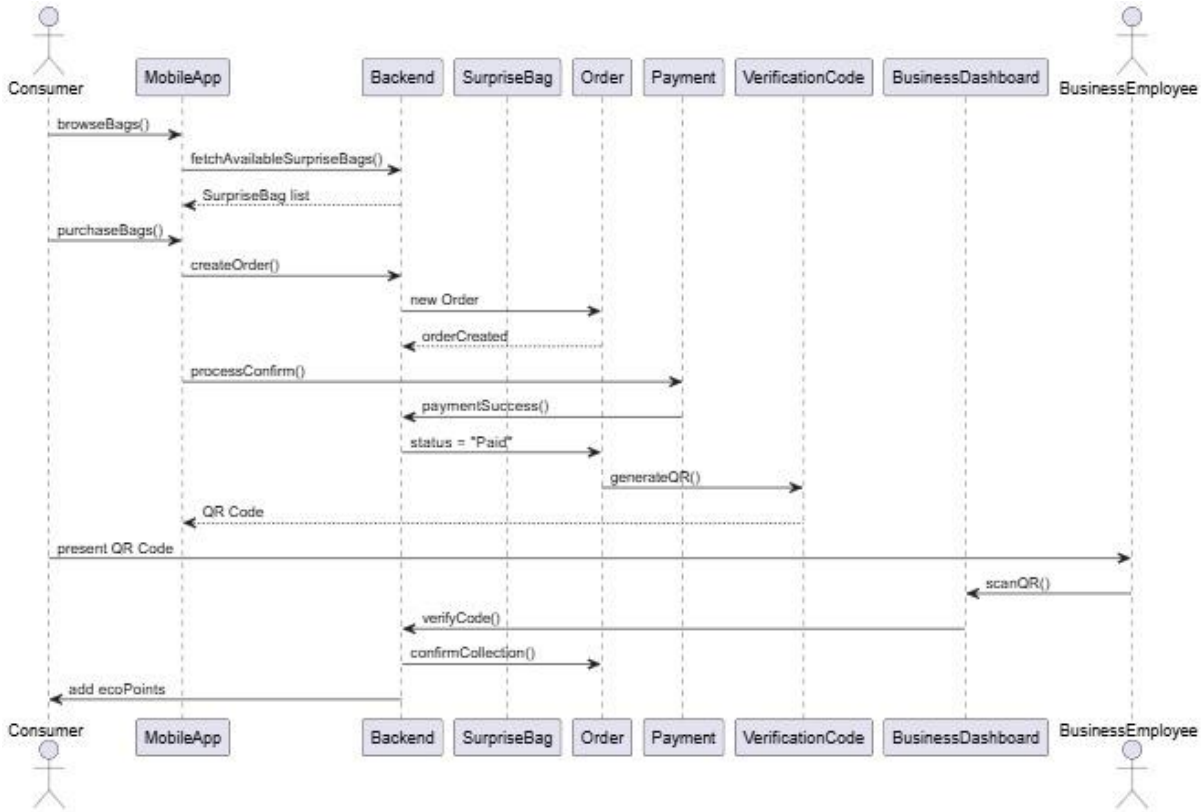

Figure 3.3.4.1 Sequence Diagram: Consumer Purchases and Collects a Surprise Bag

Sequence Steps

1. Consumer → MobileApp: browseBags()
2. MobileApp → Backend: fetchAvailableSurpriseBags()
3. Backend → MobileApp: returns list of SurpriseBag
4. Consumer selects bag → MobileApp: purchaseBags()
5. MobileApp → Backend: createOrder(Consumer, SurpriseBag)
6. Backend → Order: creates new Order
7. MobileApp → Payment: processConfirm()
8. Payment → Backend: paymentSuccess()
9. Backend updates Order.status = "Paid"
10. Order → VerificationCode: generateQR()
11. VerificationCode returns code to MobileApp
12. Consumer arrives at Business → Employee scans QR
13. BusinessDashboard → Backend: verifyCode(code)
14. Backend → Order: confirmCollection()
15. Backend → Consumer: ecoPoints.redeemPoints()

3.3.4.2 Sequence Diagram 2 : Purchase with Optional Delivery

This diagram adds courier assignment.

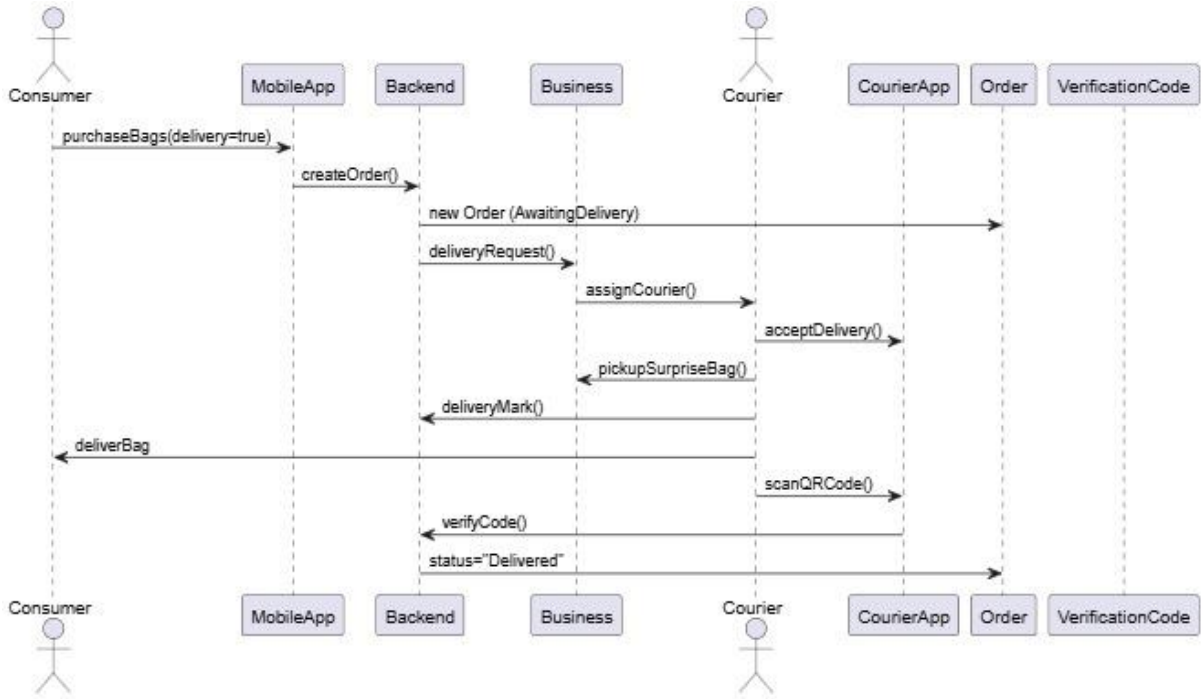

Figure 3.3.4.2 Sequence Diagram: Purchase with Optional Delivery

Sequence Steps:

1. Consumer → MobileApp: purchaseBags(delivery = true)
2. MobileApp → Backend: createOrder()
3. Backend → Order: new Order(status="AwaitingDelivery")
4. Backend → Business: notify new delivery
5. Business → Courier: assignCourier()
6. Courier → CourierApp: acceptDelivery()
7. Courier → Business: pickupSurpriseBag()
8. Courier → Backend: deliveryMark()
9. Courier delivers to Consumer
10. Courier scans QR
11. CourierApp → Backend: verifyCode()
12. Backend → Order: status = "Delivered"

3.3.4.3 Sequence Diagram 3 : Business Lists Surprise Bags

The Business employee uses the dashboard.

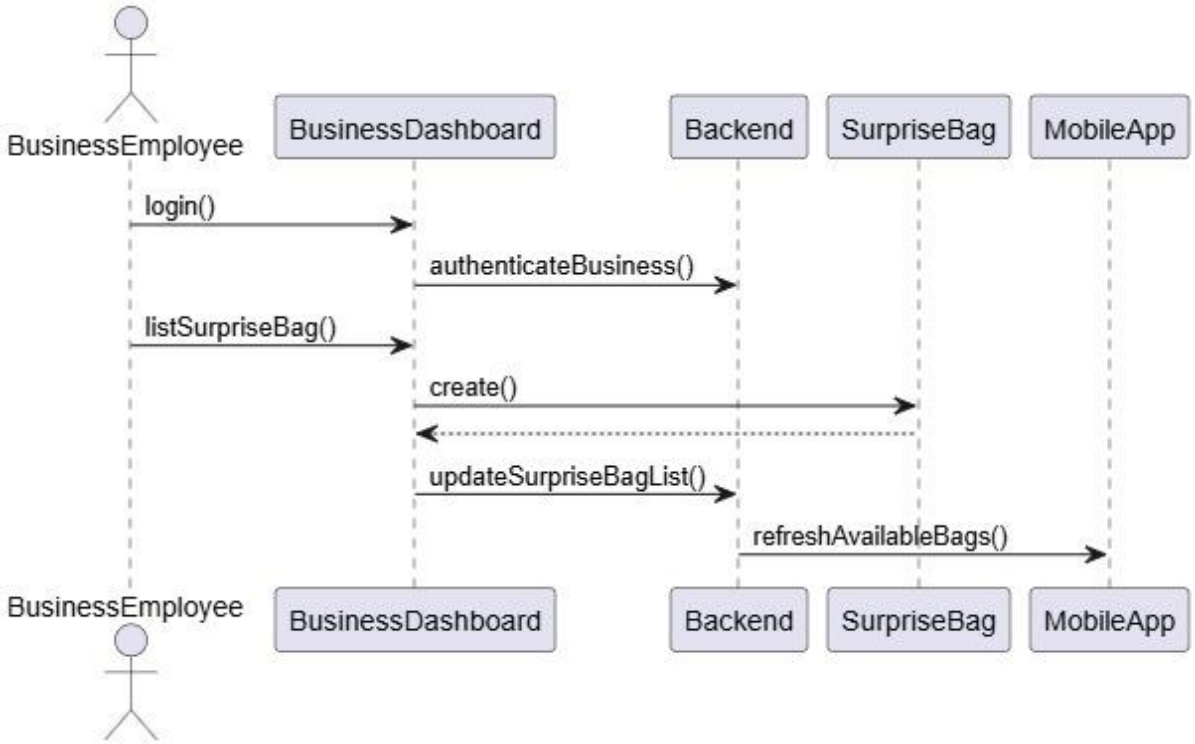

Figure 3.3.4.3 Sequence Diagram: Business Lists Surprise Bags

Sequence Steps:

1. BusinessEmployee → BusinessDashboard: login()
2. BusinessDashboard → Backend: authenticateBusiness()
3. BusinessEmployee → BusinessDashboard: listSurpriseBag()
4. BusinessDashboard → SurpriseBag: new surprise bag
5. BusinessDashboard → Backend: updateSurpriseBagList()
6. Backend → MobileApp: push updated list to consumers

3.3.4.4 Order Lifecycle State Machine

The state diagram accurately models order transitions.

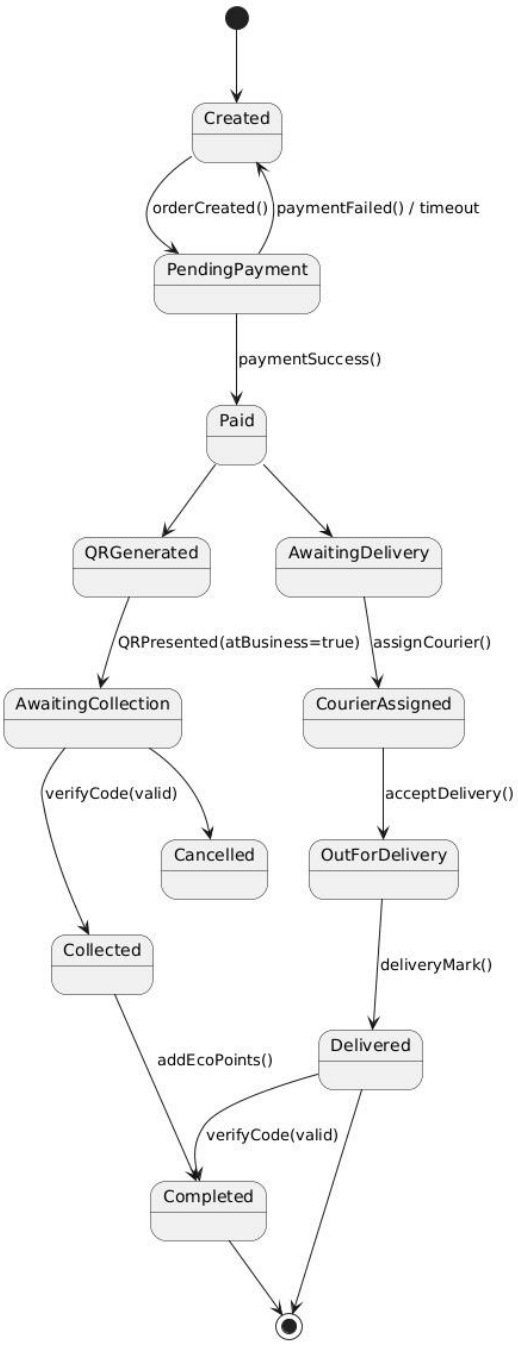

Figure 3.3.4.4 Order Lifecycle State Machine Diagram

States:

- Created
- PendingPayment
- Paid
- QRGenerated
- AwaitingCollection
- Collected
- AwaitingDelivery
- CourierAssigned
- OutForDelivery
- Delivered
- Completed
- Cancelled

Events that trigger transitions:

- orderCreated()
- paymentSuccess() / paymentFailed()
- generateQR()
- verifyCode()
- assignCourier()
- deliveryMark()
- addEcoPoints()

This state machine ensures system correctness, preventing invalid transitions like no delivery marking before pickup/delivery is validated.

3.3.5 User Interface Mock-ups

Design Tool Used: Figma

Design Link : https://www.figma.com/design/hxPJYgCxTd4cCCqegS6RAS/Untitled?node-id=0-1&t=smDSGNWMeEe0ksYq-1

Description:
The following mock-ups demonstrate the essential user interface screens for the *BagItUp!* mobile application.
These screens correspond to the critical use cases identified earlier:

- Consumer Buys and Collects a Surprise Bag
- Purchase with Optional Delivery
- Business Lists Bags

Each screen shows its visual layout, core functionality, and navigation flow between views.

Log-in Screen:

(Displays Log-in and create account options for user)

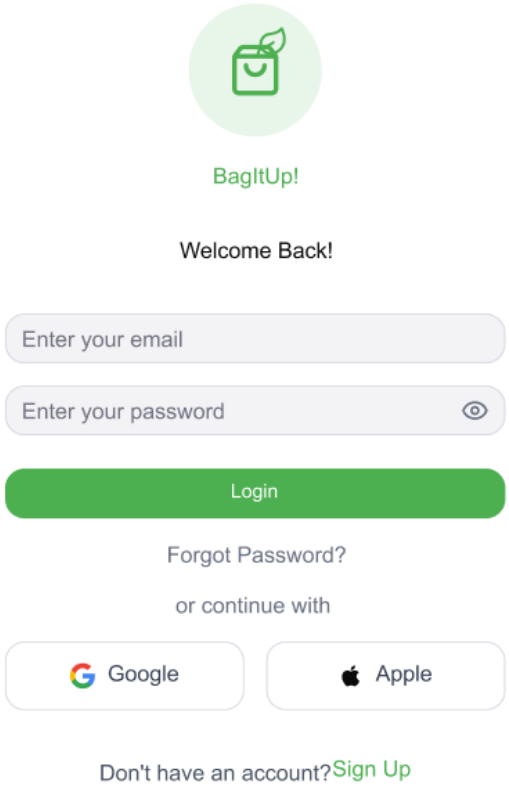

Figure 3.3.5.1 Log-in Screen Mock-up

Home / Map Screen:

(Displays nearby Surprise Bags with map pins and scrollable store list.)

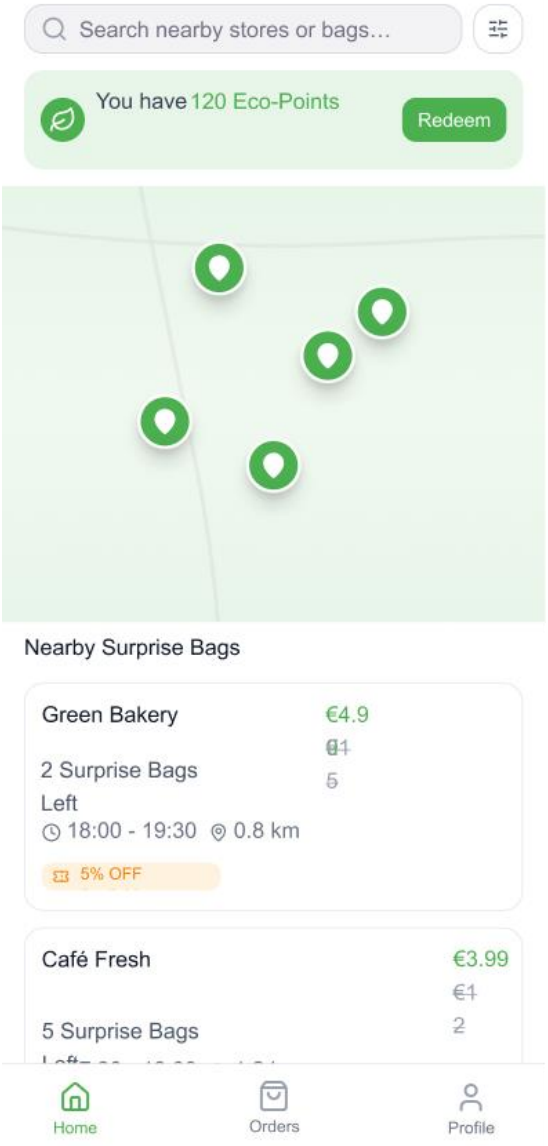

Figure 3.3.5.2 Home / Map Screen Mock-up

Bag Details Screen

(Shows bag description, price, eco-points, and options for pickup or delivery.)

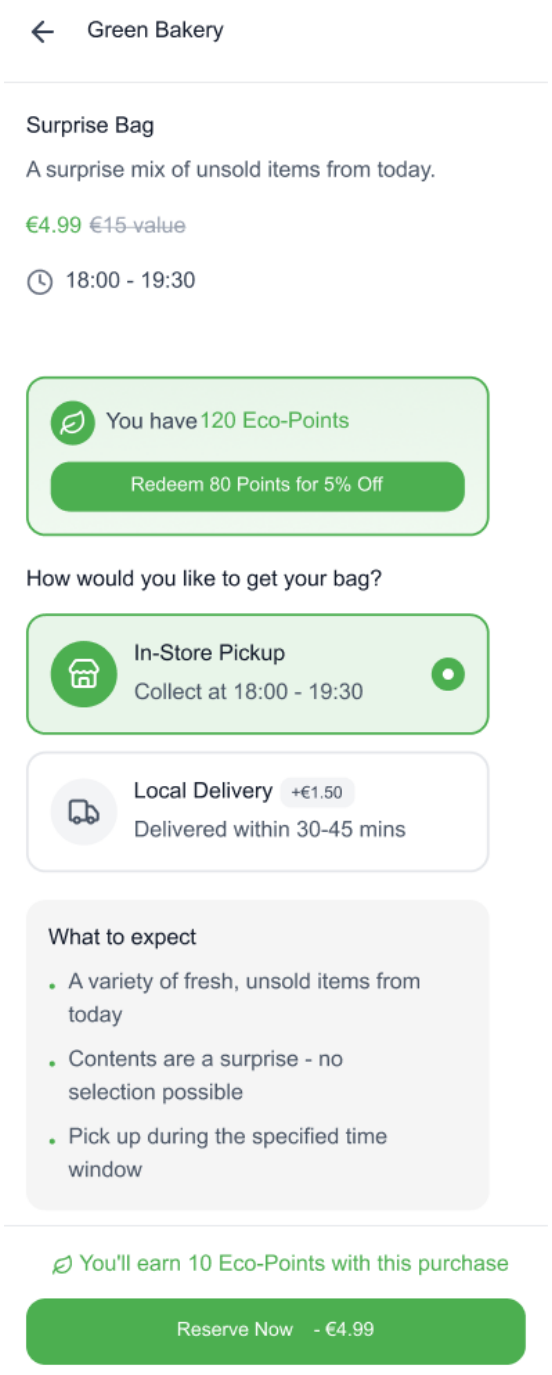

Figure 3.3.5.3 Bag Details Screen Mock-up

Payment Screen

(Processes payment and confirms order.)

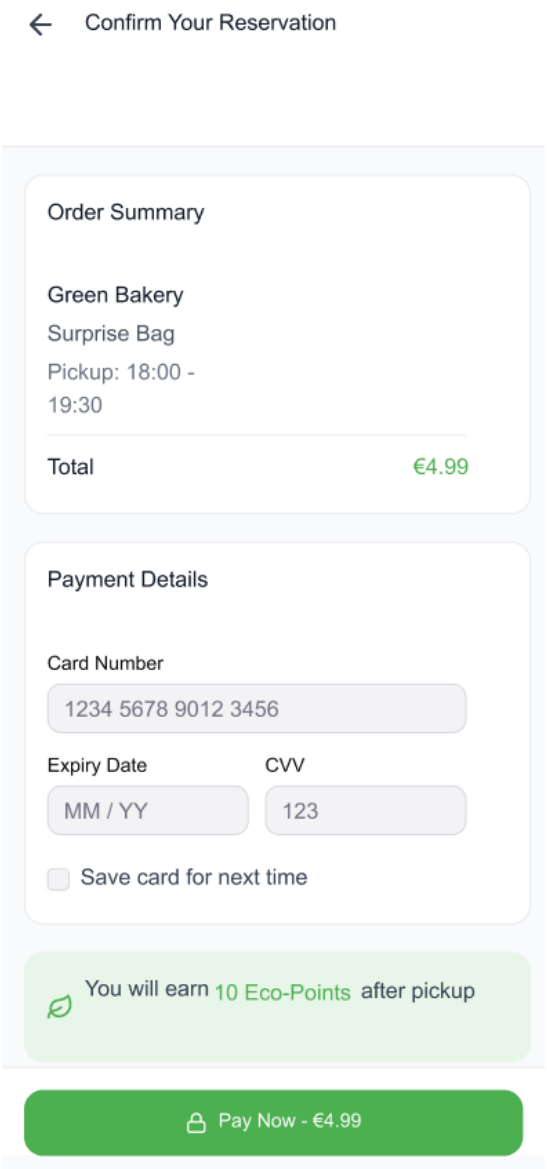

Figure 3.3.5.4 Payment Screen Mock-up

QR Code Screen

(Displays scannable code for pickup verification.)

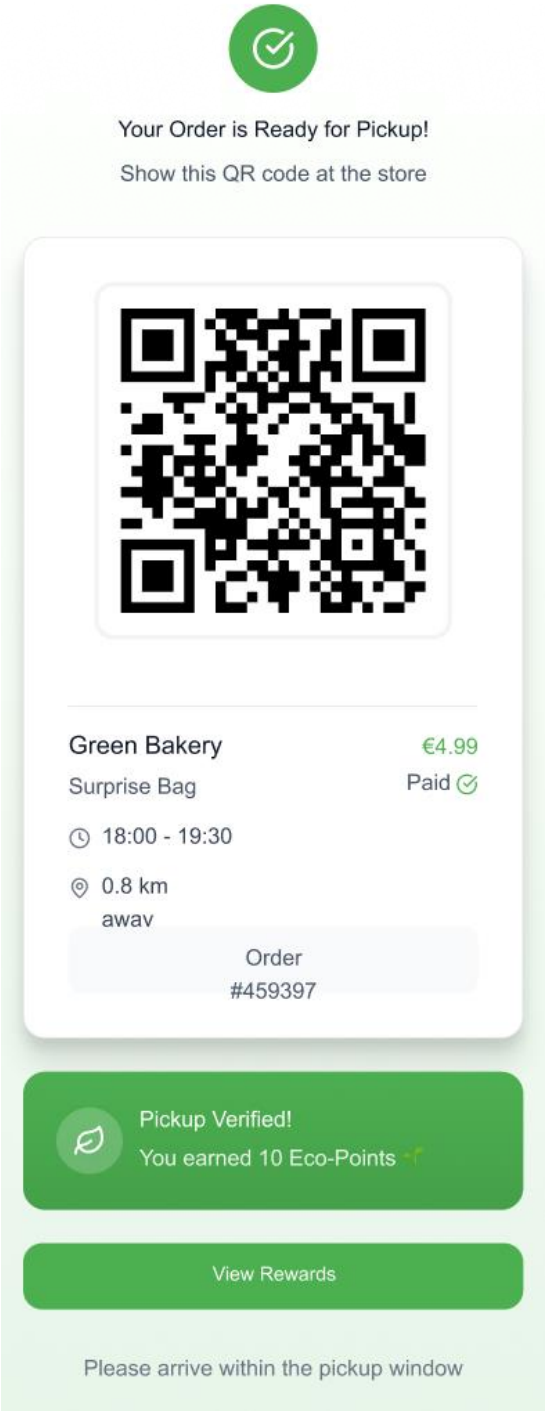

Figure 3.3.5.5 QR Code Screen Mock-up

Business Dashboard

(Allows businesses to create bags, view sales, and scan QR codes.)

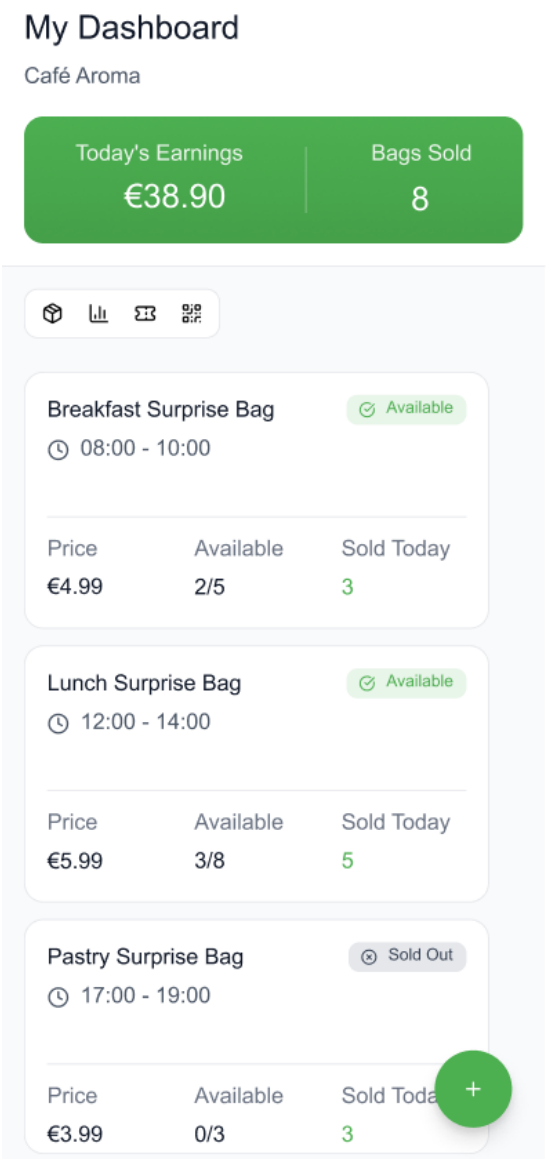

Figure 3.3.5.6 Business Dashboard Screen

Business Dashboard Campaign

(Enables businesses to create and manage promotional campaigns or coupon.)

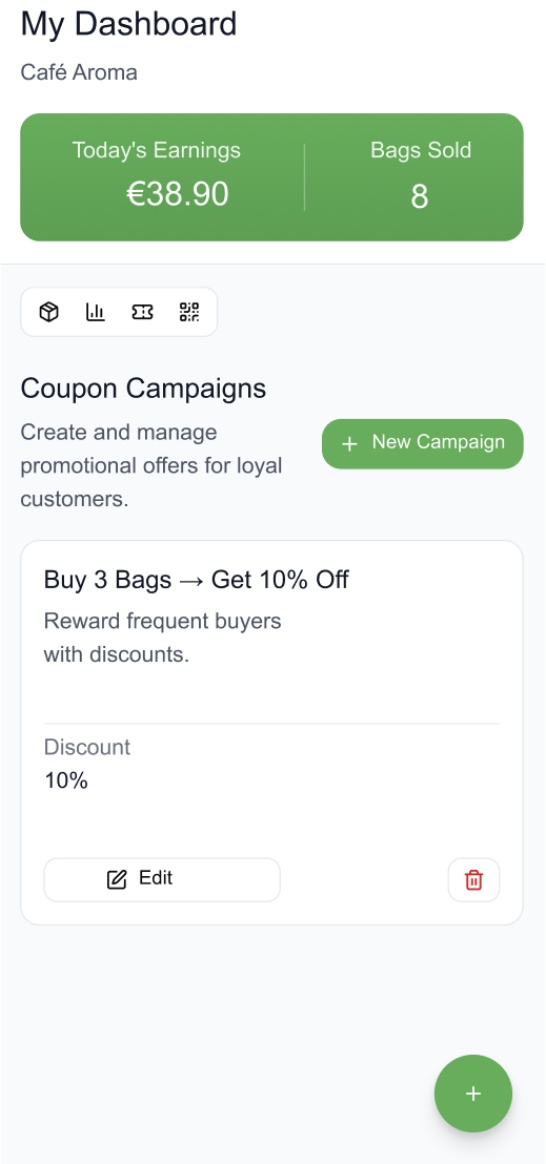

Figure 3.3.5.7 Business Dashboard Campaign Screen

Business Dashboard Add Surprise Bag

(Allows businesses to create bags and enter bag details.)

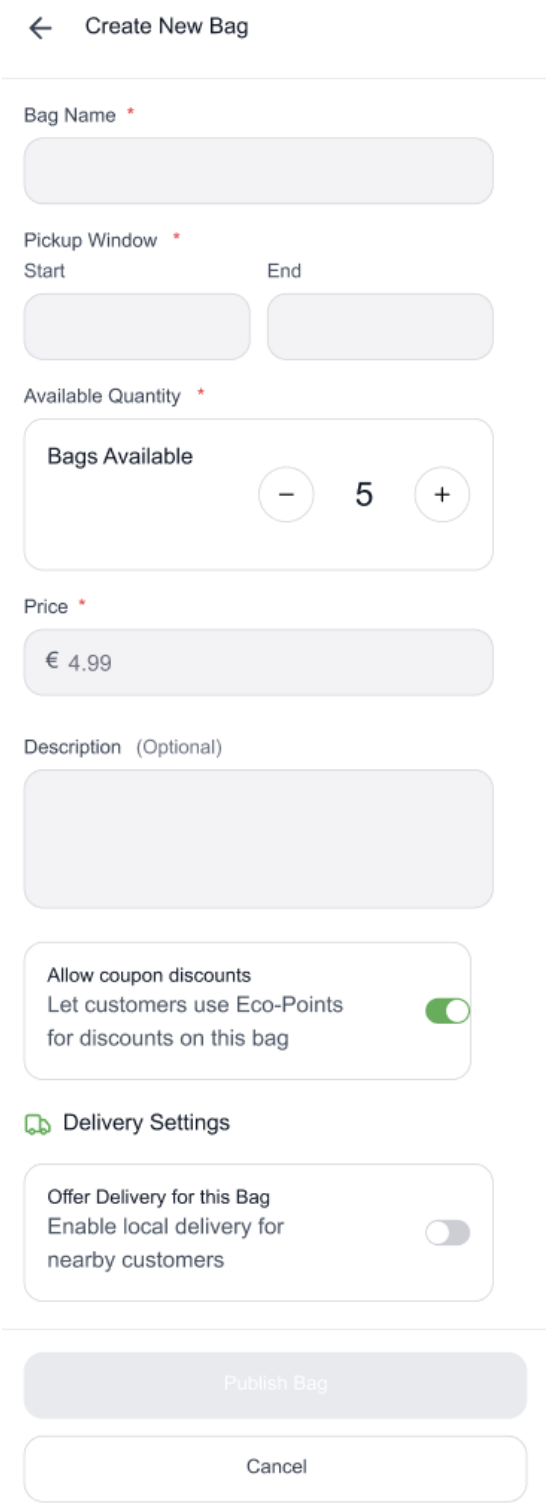

Figure 3.3.5.8 Add Surprise Bag Screen

Navigation Path:

For Consumers:

Log in → Home → Bag Details → Payment → QR Code (includes Pickup / Delivery Verification)

For Businesses:

Dashboard → Create New Bag (Enable Delivery + Allow Coupons) → Campaigns → Scan QR

4 Glossary

This section defines the key terms and concepts used throughout the BagItUp! Requirements Analysis Document. The glossary ensures consistent terminology, especially for domain-specific terms that might have different interpretations in other contexts.

Term	Definition
BagItUp!	A mobile application designed to reduce food waste by connecting consumers with businesses offering surplus food at discounted prices.
Surprise Bag	A discounted package containing surplus food items prepared by a business and sold through the BagItUp! app. The exact contents may vary daily.
Consumer (User)	The end-user who uses the BagItUp! mobile app to browse, purchase, and collect Surprise Bags.
Business(Food Provider)	Restaurants, cafes, bakeries, or other food vendors that list their surplus products on the BagItUp! platform.
Business Dashboard	A management interface used by food providers to list available Surprise Bags, approve collections, monitor sales, and create promotions.
Eco-Points	Reward points earned by consumers for each purchase, which can later be redeemed for discounts or coupons. Designed to encourage sustainable purchasing behavior.
Digital Verification Code / QR Code	A unique code generated after payment that consumers present at pickup or delivery for verification.

Term	Definition
Collection Window	The designated time frame during which consumers must collect their purchased Surprise Bags.
Local Delivery	An optional service allowing businesses to deliver Surprise Bags within a limited radius (e.g., 2–3 km).
Promotional Campaign	Marketing activity created by a business within the BagItUp! system to offer discounts or special deals to consumers.
Circular Economy	An economic system aimed at eliminating waste and reusing resources efficiently; in BagItUp!, this is achieved through redistribution of surplus food.
Sustainability	The practice of minimizing environmental impact through responsible consumption, production, and waste reduction.
Secure Pre-Payment	The process of paying in advance for a Surprise Bag using a secure, integrated digital payment system within the app.
Backend	The server-side logic of the BagItUp! system responsible for processing requests, managing data, handling payments, and coordinating interactions between mobile app, business dashboard, and internal controllers.
MobileApp	The consumer-facing application interface through which users browse Surprise Bags, make payments, view QR codes, and track eco-points.
Application Domain Objects	Core conceptual objects that represent key elements of the problem domain (e.g., Consumer, Business, SurpriseBag, Order). Used for modeling the real-world entities within the system.
Entity–Boundary–Control (EBC) Model	A modeling technique used in OOAD to classify system objects into: Entity objects (data, long-term storage) Boundary objects (interfaces) Control objects (process coordination)
State Machine Diagram	A behavioral UML model that represents the lifecycle of an object (e.g., Order), showing states and transitions triggered by events such as paymentSuccess() or verifyCode().

5 Appendix

Distribution of Work

The following table summarizes the division of responsibilities among the project team members for this stage of the project:

Member Name	Student ID	Responsibilities
Zeynep Reyyan Kaleli	220315071	Requirement analysis, preparation of Introduction and Proposed System sections, State Chart Diagram
Zahra Ismayilli	220315094	Functional and Nonfunctional Requirements documentation, proofreading, Critical Use Cases and 3 Sequence Diagrams (co-developed)
Mehmet Mert Yiğit	220315103	System modeling (Use Cases), Critical Use Cases and 3 Sequence Diagrams (co-developed)
Begüm Sezer	220315067	User Interface mock-up design, Report editing, Entity–Boundary–Control Model List (co-developed)
Göktuğ Kayacı	230315009	Use Case Model diagram, Glossary and Appendix preparation, overall editing, UML Class Diagram
Aykhan Bakhshizada	210315091	Coordination, meeting organization, and document formatting, Entity–Boundary–Control Model List (co-developed)

All members contributed equally to discussions, design decisions, and documentation preparation to ensure balanced participation and comprehensive system analysis.

Annex – II

Meeting Minutes

Date:	25.10.2025
Location:	Manisa Celal Bayar University
Duration:	1.5 hours
Participants:	All group members

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

Agenda:

- Define the project topic and problem domain
- Conduct initial brainstorming on requirements and features
- Assign early-stage responsibilities and deliverables

Discussion Summary:

The team initiated the first project meeting by confirming the selected topic: BagItUp! – A Sustainable Food Waste Reduction System. Members discussed the increasing issue of food waste and reviewed existing applications such as Too Good To Go and Olio to identify gaps and potential enhancements.

Key system objectives were defined, including:

- Minimizing food waste through surplus redistribution
- Providing a convenient, location-based discovery system
- Ensuring secure pre-payment and QR-based collection verification
- Encouraging sustainability through an eco-points reward system

Preliminary functional requirements were outlined (FR-01 to FR-08), covering bag listing, secure payment, and loyalty features. The nonfunctional requirements such as performance, security, and usability were also identified.

Decisions Made:

- Project title finalized as BagItUp!
- UI design tool selected: Figma
- Shared workspace created on Google Drive for collaboration
- Next meeting scheduled to refine system models and finalize the RAD document structure.

Annex – II

Meeting Minutes

Date:	05.11.2025
Location:	Manisa Celal Bayar University
Duration:	2 hours
Participants:	All group members

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

Agenda:

- Review and finalize written sections of the RAD document
- Validate Functional and Nonfunctional Requirements
- Review Use Case and Object Models
- Discuss pending Project Implementation stages

Discussion Summary:

The team reviewed all completed sections of the report:

- Introduction, Current System, and Proposed System were finalized, incorporating clear objectives, system scope, and success criteria.
- Functional and Nonfunctional Requirements were refined to ensure measurable and technically feasible targets.
- System Models, including Use Case, were discussed and approved with minor revisions.
- Glossary and Appendix sections were completed to standardize terminology and document contributions.
- UI Mock-ups created in Figma were reviewed and linked in the RAD.

It was noted that the Project Implementation phase remain incomplete and will be developed in the next iteration of the project.

Decisions Made:

- Approved all completed sections for submission.
- Agreed to schedule a final meeting after completion of Implementation design.
- Confirmed formatting consistency and cross-referencing within the final document.

Annex – II

Meeting Minutes

Date:	03.12.2025
Location:	Manisa Celal Bayar University
Duration:	1 hour
Participants:	All group members

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

Agenda:

- Finalize Object Model
- Assign responsibilities for UML diagrams
- Review EBC (Entity–Boundary–Control) classification

Discussion Summary:

The team refined the class diagram based on instructor feedback and ensured alignment with all functional requirements. Responsibilities for the UML Class Diagram, State Machine, and Sequence Diagrams were formally assigned. The EBC model was drafted, identifying boundary and control objects used in the Dynamic Model section.

Decisions Made:

- Göktuğ completes UML Class Diagram.
- Zeynep prepares the Order Lifecycle State Machine.
- Zahra & Mert finalize 3 Sequence Diagrams.
- Ayhan prepares complete EBC list.
- Begüm updates Application Domain Objects. Updates the entity-boundry-control objects according to the diagrams.

Annex – II

Meeting Minutes

Date:	05.12.2025
Location:	Manisa Celal Bayar University
Duration:	1.5 hours
Participants:	All group members

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

Agenda:

- Dynamic Models final revision
- UI screenshots integration
- Glossary & Appendix finalization

Discussion Summary:

The team reviewed all sequence diagrams and validated message flows with object responsibilities. UI mock-ups were finalized in Figma and correctly renamed according to the numbering schema. Glossary items were expanded to include domain modeling terms. Appendix was updated with the final distribution of work.

Desicions Made:

- Add final figure numbering aligned with teacher requirements.
- Insert updated glossary definitions related to class diagrams, state charts, and domain objects.
- Prepare document for final submission.